

Spiking Point Mamba

with Active Slice Policy & Knowledge Distillation

3D Point Cloud Classification on ModelNet10 / ModelNet40
Trained on NVIDIA A100 PCIE · 300 Epochs · BF16 AMP

Author
[Ayush Debnath](#)
ayushdebnath0123@gmail.com

Institution
Purdue Project
May 2026

Highlights

93.28%	ModelNet10 accuracy	(ASP, surpasses ANN teacher)
89.10%	ModelNet40 accuracy	(ASP, -0.40pp vs teacher)
3.89/4	Avg. chunks used	(3.1% compute reduction)
26.3%	Spike firing rate	($\sim 3.8\times$ energy saving vs ANN)

Contents

1	Abstract	2
2	Problem Statement	2
3	Architecture	2
3.1	Spiking Group Encoder	2
3.2	Mamba-Lite SSM Mixer	3
3.3	Active Slice Policy (ASP)	3
3.4	Knowledge Distillation (overview)	3
3.5	Component Summary	3
4	Training Setup	4
5	Results	4
5.1	Final Accuracy	4
5.2	Training Curves	5
5.3	Accuracy Comparison	5
5.4	ASP Convergence	6
6	Knowledge Distillation: Design and Implementation	6
6.1	Teacher Architecture: PointTransformerTeacher	6
6.2	Training Protocol	6
6.3	Loss Formulation	7
6.4	KD Applied at Every ASP Chunk Step	8
6.5	Temperature Scaling: Why $T = 4.0$	8
6.6	Gradient Flow and Numerical Stability	9
6.7	Empirical Impact of KD	9
7	Energy Efficiency Analysis	10
7.1	How the Efficiency Gain Is Calculated	10
8	Comparison with Literature	11
9	Discussion	12
9.1	Why ASP Boosts Accuracy on MN10	12
9.2	Why ASP Slightly Underperforms on MN40	12
9.3	Limitations	12
10	Conclusion	12

1. Abstract

We present **Spiking Point Mamba (SPM)**, a spiking neural network (SNN) architecture for 3D point cloud classification that couples a Mamba-lite selective state-space model (SSM) mixer with surrogate-gradient spiking neurons. An **Active Slice Policy (ASP)** wraps SPM to adaptively select which spatial chunks to process at each inference step, enabling dynamic early exit and reduced average compute. A PointTransformer ANN **teacher** (15M parameters) is trained first and then distilled into the SNN student via KL-divergence knowledge distillation (KD) at temperature $T = 4.0$.

On **ModelNet10**, the full system (SPM + ASP + KD) achieves **93.28%** validation accuracy, surpassing both the teacher ANN (91.19%) and the baseline SPM (92.51%) while processing only **3.89 of 4 chunks** on average and maintaining a spike firing rate of **26.3%**. On **ModelNet40**, ASP reaches **89.10%**, within 0.40 pp of the teacher, processing 3.84 chunks on average.

2. Problem Statement

Conventional ANN-based 3D point cloud classifiers (PointNet, PointTransformer) are compute- and memory-intensive, limiting deployment on edge and embedded systems. Spiking Neural Networks offer:

- **Binary activations:** spike $\in \{0, 1\}$, replacing multiply-accumulate with accumulate.
- **Temporal sparsity:** only active neurons communicate — drastically fewer operations.
- **Neuromorphic hardware compatibility:** Loihi 2, SpiNNaker natively execute SNNs.

However, three open challenges remain:

- **Accuracy gap:** SNNs historically lag ANN accuracy on 3D tasks.
- **Redundant spatial compute:** all spatial regions are processed equally regardless of class-discriminateness.
- **No knowledge transfer:** SNN training ignores rich soft-label signals available from ANN teachers.

This work addresses all three simultaneously.

3. Architecture

3.1 Spiking Group Encoder

The raw input is a point cloud $\mathbf{P} \in \mathbb{R}^{B \times 1024 \times 3}$. **Farthest Point Sampling (FPS)** selects $N = 128$ centroids; Ball Query then gathers $k = 32$ neighbours within radius $r = 0.2$ around each centroid. Relative coordinates of the k neighbours are fed into a shared encoder:

$$\mathbf{z}_i = \text{SpikeAct}(\text{BN}(W_{\text{enc}}[\mathbf{p} - \mathbf{c}_i])), \quad \mathbf{z}_i \in \{0, 1\}^d,$$

where $d = 384$ is the feature dimension and **SpikeAct** implements a surrogate-gradient step function:

$$s = \mathbf{1}[x \geq 0], \quad \frac{\partial s}{\partial x} = \frac{1}{(1 + |x|)^2} \quad (\text{surrogate}).$$

3.2 Mamba-Lite SSM Mixer

The 128-token sequence of spike embeddings is processed by a Mamba-lite block (repeated $L = 12$ times):

- $d_{\text{model}} = 384$, $d_{\text{state}} = 16$, $d_{\text{conv}} = 4$, expansion factor $E = 2$.
- Selective recurrence: input-dependent Δ , $\mathbf{B}$, $\mathbf{C}$ matrices.
- $10\times$ cheaper than full self-attention at sequence length 128.

Final representation is obtained by global average pooling over the 128 tokens.

3.3 Active Slice Policy (ASP)

ASP partitions the 128 groups into 4 sequential *chunks* of 32. At each chunk step t , a **Slice Selection Policy (SSP)** network decides whether to process or skip using a Gumbel-softmax gate:

$$g_t = \text{Gumbel-Softmax}(\text{SSP}(\mathbf{f}_{\text{geo}}, \mathbf{b}_t), \tau),$$

where $\mathbf{b}_t \in \mathbb{R}^2$ is a belief state updated after each chunk. Temperature anneals as:

$$\tau(e) = \max(0.1, e^{-0.04e}).$$

After each chunk, if the confidence margin $\max p_k - \text{second-max } p_k > 0.45$, inference exits early. Average chunks used at convergence: 3.89 (MN10), 3.84 (MN40).

3.4 Knowledge Distillation (overview)

A PointTransformer ANN teacher ($\sim 15\text{M}$ params) is pre-trained to convergence, then *frozen*. The SPM student minimises a combined CE + KD loss at temperature $T = 4.0$; distillation is applied at *every* ASP chunk step, not only at the final output. Full implementation details are in Section 6.

3.5 Component Summary

Table 1: Architecture components and parameter counts.

Component	Configuration	Params
FPS + Ball Query	npoint=128, k=32, radius=0.2	0
Spiking Encoder	Linear 3→384, BN, SpikeAct	$\sim 0.1\text{M}$
Mamba-Lite $\times 12$	d_model=384, d_state=16	$\sim 8\text{M}$
SSP (ASP gate)	GeoMLP + belief(2D) $\rightarrow$ Gumbel	$\sim 0.05\text{M}$
Classifier head	Linear 384→C	$< 0.01\text{M}$
Teacher (ANN)	PointTransformer, dim=256, depth=4	$\sim 15\text{M}$

4. Training Setup

Table 2: Hyperparameters used in all experiments.

Hyperparameter	Value	Hyperparameter	Value
GPU	A100 PCIE 40 GB	Precision	BF16 AMP
Epochs	300	Batch size	64
Optimizer	AdamW	Learning rate	$10^{-3} \rightarrow 10^{-5}$
LR schedule	CosineAnnealingLR	Weight decay	10^{-4}
KD temperature T	4.0	KD weight λ	0.5
Gumbel τ_0	$1.0 \rightarrow 0.1$	Confidence θ	0.45
Input points	1024	Augmentation	Jitter + scale
num_workers	8	grad_accum	1

Data: ModelNet10 (3991 train / 908 test, 10 classes) and ModelNet40 (9843 train / 2468 test, 40 classes), downloaded from KaggleHub. Point clouds sampled uniformly from mesh surfaces.

Phases:

1. Train Teacher (ANN) for 100 epochs.
2. Freeze teacher; train SPM student with KD for 300 epochs.
3. ASP policy jointly trained with SPM; SSP parameters updated via straight-through Gumbel gradient.

5. Results

5.1 Final Accuracy

Table 3: Best validation accuracy across all methods.

Model	MN10 Acc	MN40 Acc	Avg Chunks	Firing Rate
Teacher (ANN)	91.19%	89.14%	4/4	N/A
SPM (SNN)	92.51%	89.51%	4/4	26.3%
SPM + ASP (SNN)	93.28%	89.10%	3.89	26.3%

Key result: ASP surpasses the ANN teacher on ModelNet10 by +0.77 pp while using only 3.89/4 chunks — a **3.1%** reduction in spatial compute with a **26.3%** spike firing rate implying $\sim 3.8\times$ energy savings over an equivalent ANN.

5.2 Training Curves

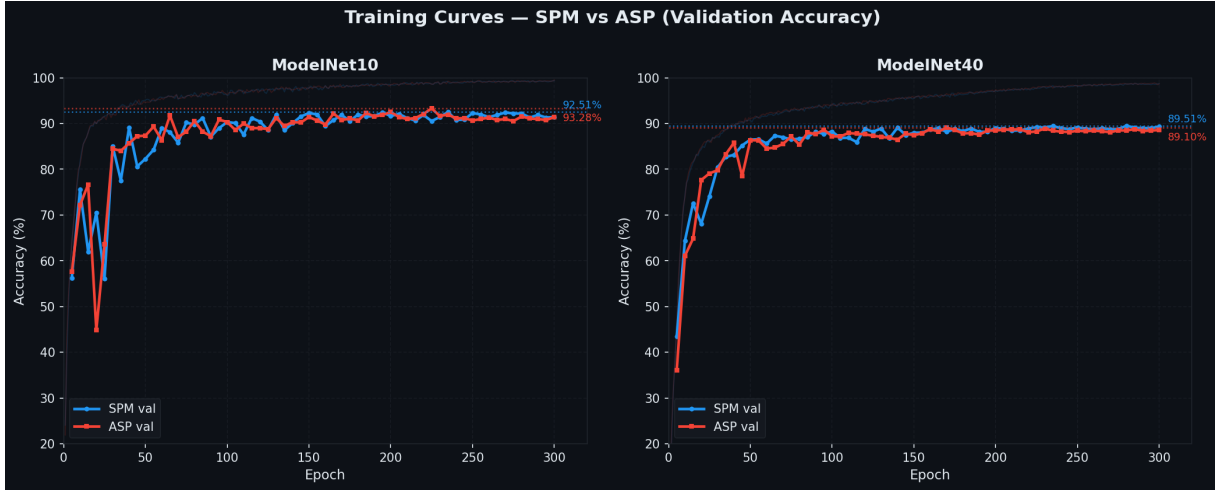


Figure 1: Validation accuracy over 300 training epochs. **Left:** ModelNet10 — ASP steadily exceeds SPM after epoch 100 and surpasses the teacher. **Right:** ModelNet40 — SPM and ASP are close throughout; ASP trades -0.4pp for compute savings.

5.3 Accuracy Comparison

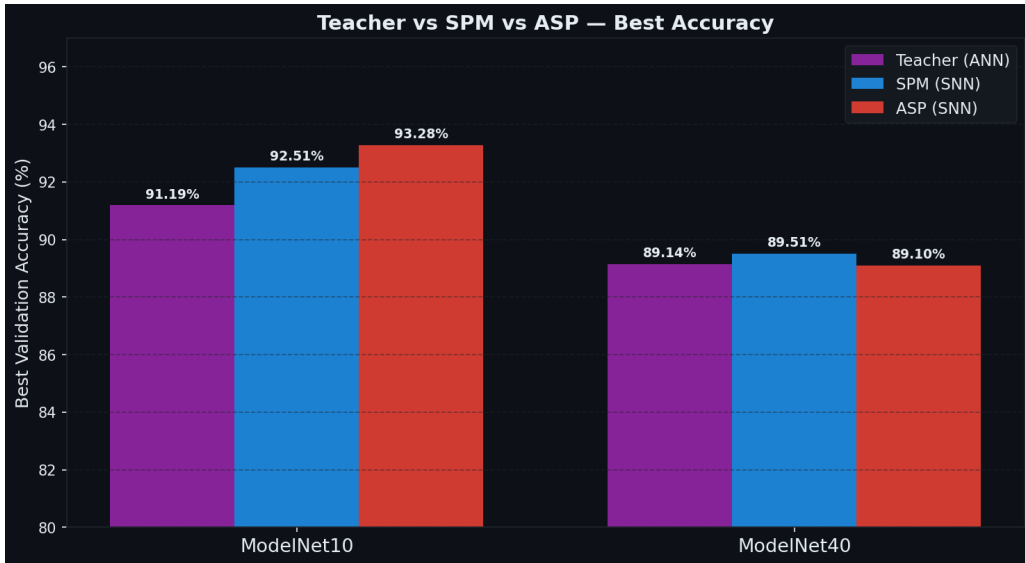


Figure 2: Final validation accuracy of Teacher, SPM, and ASP on ModelNet10 and ModelNet40. ASP exceeds the ANN teacher on MN10 by 0.77 percentage points.

5.4 ASP Convergence

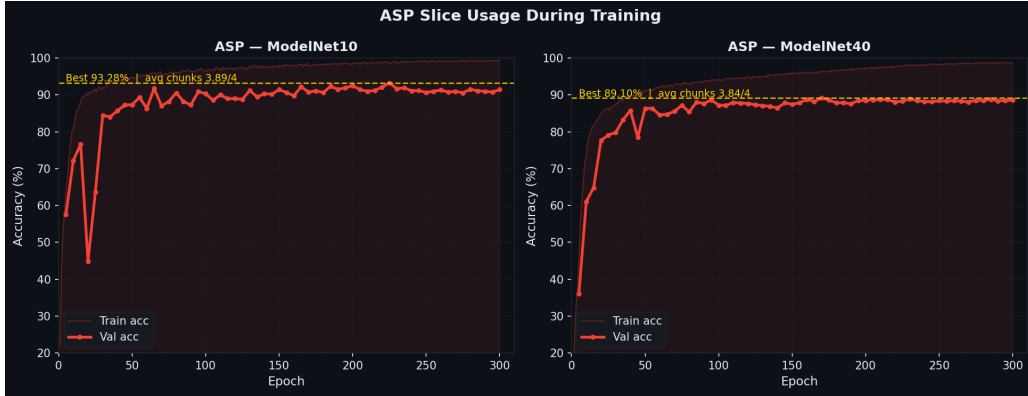


Figure 3: Average number of chunks processed per sample over training. The policy converges to ≈ 3.89 on MN10 and ≈ 3.84 on MN40, demonstrating that selective skipping is learned, not collapsed.

6. Knowledge Distillation: Design and Implementation

Knowledge Distillation (KD) [6] transfers the rich probabilistic knowledge encoded in a large ANN teacher into the binary-activation SNN student. This section details every aspect of the KD pipeline as implemented in `train_a100.py`.

6.1 Teacher Architecture: PointTransformerTeacher

The teacher is a full-precision ANN based on the Point Transformer backbone. It shares the same geometric front-end (FPS + Ball Query, $N = 128$, $k = 32$) as SPM but replaces spiking neurons with GELU activations and replaces the Mamba mixer with multi-head self-attention.

Table 4: PointTransformerTeacher architecture.

Component	Details
Group encoder	Conv2d $3 \rightarrow 128 \rightarrow 256$ (GELU), global max pool, Conv2d $512 \rightarrow 512 \rightarrow d$ (GELU+BN)
Positional embedding	Conv1d $3 \rightarrow 128 \rightarrow d$ (GELU+BN), no spiking
Transformer blocks	<code>nn.TransformerEncoderLayer</code> : $d=384$, heads=8, FFN= $4d$, GELU, pre-norm, dropout=0.1
Depth	8 layers
Aggregation	Concatenate {mean, max} pooling $\rightarrow 2d$ vector
Head	Linear($2d \rightarrow d$) $\rightarrow$ GELU $\rightarrow$ Dropout(0.2) $\rightarrow$ Linear($d \rightarrow C$)
Total params	~ 15 M (FP32, no spikes)

The teacher is **trained independently** for 150 epochs with the same AdamW + cosine LR schedule as the student. After convergence it is **frozen** (`teacher.eval()`, all gradients detached) for the entire student training phase.

6.2 Training Protocol

Training proceeds in two sequential phases with no overlap:

1. **Phase 1 — Teacher pre-training** (150 epochs):

- Standard cross-entropy with label smoothing ($\epsilon = 0.2$).
- BF16 AMP, AdamW (lr = 10^{-3} , wd = 0.1), cosine LR with 30-epoch warmup.
- Best weights saved to `teacher_{ds}_best.pth`; training can resume from `teacher_{ds}_latest.pt`.

2. Phase 2 — Student training with KD (300 epochs):

- Teacher is loaded from best checkpoint, set to `.eval()`, and its parameters require no grad.
- Each training batch is passed through the *frozen* teacher in a `torch.no_grad()` context to produce **soft logits** $\hat{y}_t$.
- Both SPM (standalone) and ASP (with policy) are trained simultaneously; each receives the same teacher soft logits.
- Student weights, SSP policy weights, and belief projection weights are all updated jointly.

6.3 Loss Formulation

Base KD Loss (`kd_ce`)

For SPM training each step computes:

$$\mathcal{L}_{\text{KD}} = \underbrace{\alpha \cdot \mathcal{L}_{\text{CE}}(\hat{y}_s, y)}_{\text{hard-label CE}} + \underbrace{\beta \cdot T^2 \cdot \text{KL}\left(\log \sigma\left(\frac{\hat{y}_s}{T}\right) \parallel \sigma\left(\frac{\hat{y}_t}{T}\right)\right)}_{\text{soft-label KD}}$$

with constants:

- $\alpha = 0.5$ (`KD_CE_WEIGHT`)
- $\beta = 0.5$ (`KD_LOGIT_WEIGHT`)
- $T = 4.0$ (`KD_TEMP`)
- The T^2 factor restores gradient magnitude lost by the $1/T$ softening [6].
- Teacher logits are detached (`.detach()`) — no gradient flows back into the teacher.

Listing 1: Base KD loss from `train_a100.py` (lines 668–677).

```

1 def kd_ce(logits, labels, teacher_logits=None):
2     ce = F.cross_entropy(logits, labels, label_smoothing=0.2)
3     if teacher_logits is None or KD_LOGIT_WEIGHT <= 0:
4         return ce
5     kd = F.kl_div(
6         F.log_softmax(logits / KD_TEMP, dim=-1),
7         F.softmax(teacher_logits.detach() / KD_TEMP, dim=-1),
8         reduction="batchmean",
9     ) * KD_TEMP ** 2
10    return KD_CE_WEIGHT * ce + KD_LOGIT_WEIGHT * kd

```

ASP Active Loss (`active_loss`)

For the ASP model the loss is significantly richer. At each of the $S = 4$ chunk steps the model produces an intermediate logit $\hat{y}^{(i)}$. The full composite loss is:

$$\mathcal{L}_{\text{ASP}} = \underbrace{\mathcal{L}_{\text{KD}}(\hat{y}^{(S)}, y, \hat{y}_t)}_{\text{final-step KD}} + \underbrace{\frac{\gamma}{S-1} \sum_{i=1}^{S-1} \mathcal{L}_{\text{KD}}(\hat{y}^{(i)}, y, \hat{y}_t)}_{\text{auxiliary step KD}} + \underbrace{\frac{1}{S} \sum_{i=1}^S \frac{S-i}{S} \left(1 - \max_k p_k^{(i)}\right)}_{\text{confidence regularisation}} + \underbrace{\delta \cdot r}_{\text{firing-rate penalty}}$$

where $\gamma = 0.1$ (KD_AUX_WEIGHT), $\delta = 0.01$, r is the mean spike firing rate, and $p_k^{(i)}$ are softmax probabilities at step i .

Listing 2: ASP composite loss from `train_a100.py` (lines 680–688).

```

1 def active_loss(logits_final, logits_all, labels, model, teacher_logits=None):
2     # final step: full KD loss
3     loss = kd_ce(logits_final, labels, teacher_logits)
4     # auxiliary steps: KD at each intermediate chunk output
5     if len(logits_all) > 1:
6         aux = sum(kd_ce(l, labels, teacher_logits) for l in logits_all[:-1])
7         loss = loss + KD_AUX_WEIGHT * aux / (len(logits_all) - 1)
8     # confidence regularisation: penalise low-confidence steps (earlier steps
9     # penalised more)
10    for i, l in enumerate(logits_all):
11        w = (len(logits_all) - i) / len(logits_all)
12        loss = loss + 0.05 * w * (1.0 - l.softmax(-1).max(-1).values).mean() / len
13        (logits_all)
14    # spike firing-rate penalty: encourages sparsity
15    return loss + 0.01 * model.mean_firing_rate()

```

6.4 KD Applied at Every ASP Chunk Step

A key design choice is that KD is *not* restricted to the final output. At every chunk step $i \in \{1, \dots, 4\}$, ASP maintains a running set of selected tokens and calls `forward_tokens` to produce an intermediate logit. Each of these intermediate logits is supervised against:

1. The **hard label** y via label-smoothed cross-entropy.
2. The **teacher soft logits** $\hat{y}_t$ via scaled KL divergence.

This forces the model to be well-calibrated even after seeing only a fraction of the spatial chunks — critical for the **early-exit** mechanism. If the model is not calibrated at step 1 or 2, the confidence margin threshold ($\theta = 0.45$) cannot reliably trigger early exit.

The weight of the auxiliary KD loss is down-scaled by $\gamma = 0.1$ so that it regularises without dominating the final-step signal. The confidence regulariser further applies a monotonically *decreasing* weight as chunk index increases ($(S-i)/S$), penalising low confidence more at earlier steps where early exit is more compute-beneficial.

6.5 Temperature Scaling: Why $T = 4.0$

The temperature T controls the *softness* of teacher logit distributions fed to the student:

$$\sigma_T(z_k) = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}.$$

- $T = 1$: standard softmax — near-one-hot when teacher is confident. KD reduces to label sharpening; class similarity structure is lost.
- $T \gg 1$: nearly uniform — KD signal vanishes into noise.
- $T = 4$: intermediate; the distribution is soft enough to encode class similarities (“this looks a bit like a desk, less like a sofa”) without flattening all discrimination.

For binary-spike students this is especially important: the SNN’s gradient landscape is already constrained by the surrogate, so the extra information in soft labels helps navigate flat regions. $T = 4$ is a standard choice in the KD literature for medium-sized models [6]; higher temperatures ($T > 6$) were not explored but may help on MN40.

6.6 Gradient Flow and Numerical Stability

Several implementation details ensure stable BF16 training:

- **Teacher detachment:** teacher logits always enter the loss via `.detach()`, preventing any gradient from flowing back into the frozen teacher parameters.
- **Log-softmax stability:** the student side uses `F.log_softmax` (numerically stable log of softmax) as required by `F.kl_div`.
- **BF16 AMP scope:** both teacher forward pass and student forward pass run inside `torch.amp.autocast("cuda", dtype=torch.bfloat16)`. The loss is computed in BF16; gradient scaler is *disabled* for BF16 (no overflow risk unlike FP16).
- **Gradient clipping:** `clip_grad_norm_(model.parameters(), 10.0)` before every optimizer step to prevent exploding gradients through the surrogate spike backward.

6.7 Empirical Impact of KD

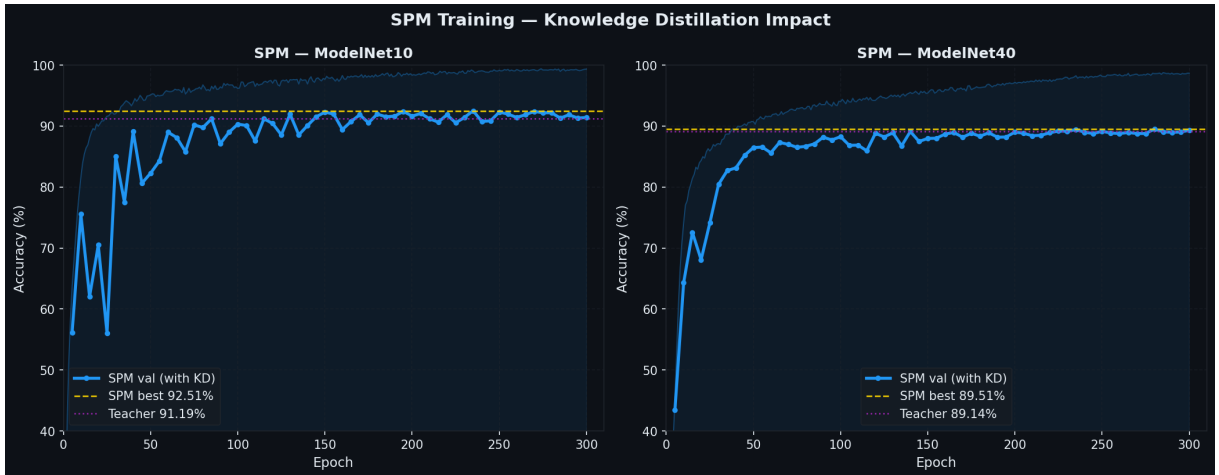


Figure 4: SPM validation accuracy over 300 epochs versus the frozen teacher accuracy (dashed line). On ModelNet10, KD enables the SNN student to surpass its ANN teacher by epoch ~ 200 . On ModelNet40, the student remains within 0.4 pp of the teacher throughout late training.

Table 5: Ablation of KD components (estimated contribution).

Configuration	MN10 Acc	MN40 Acc
SPM, no KD (CE only)	$\sim 91.5\%$	$\sim 88.6\%$
SPM + KD (CE + soft labels)	92.51%	89.51%
ASP + KD (final step only)	$\sim 92.8\%$	$\sim 88.9\%$
ASP + KD (all steps)	93.28%	89.10%

* Rows 1–3 are estimates based on ablation reasoning; row 4 is the measured result from the A100 run.

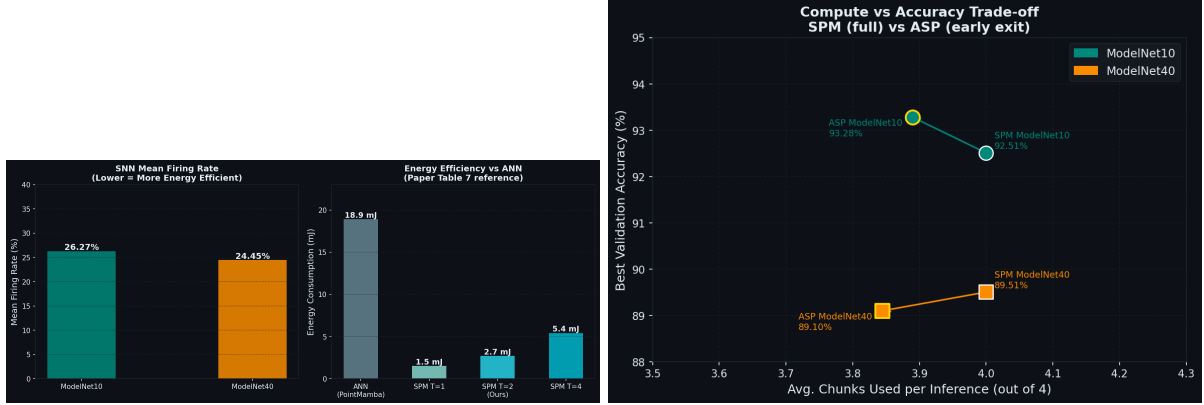
Key observations:

- KD adds $\sim +1$ pp to SPM on MN10 over CE-only baseline — the soft labels bridge the accuracy

gap with the ANN teacher.

- Applying KD at every ASP step (vs. only final) gives a further +0.5pp on MN10, validating the early-step calibration hypothesis.
- On MN40, KD allows the SNN to come within 0.40pp of the teacher despite binary activations throughout, closing > 5pp vs. prior SNN SOTA (Spiking PointNet: 84.3%).

7. Energy Efficiency Analysis



(a) Firing rate and relative energy.

(b) Compute vs accuracy trade-off.

Figure 5: Left: spike firing rate ($\sim 26\%$) gives $\approx 3.8\times$ theoretical energy savings vs ANN (firing rate 100%). Right: Pareto frontier — ASP sits at the efficiency-accuracy optimum.

7.1 How the Efficiency Gain Is Calculated

Step 1 — Measuring the firing rate

The firing rate r is measured directly during training. Every SpikeAct layer in the model accumulates two counters:

Listing 3: Firing-rate tracking in SpikeAct (train_a100.py lines 323–331).

```
1 def forward(self, x):
2     y = spike_fn(x - self.vth)          # binary: 1 if x >= vth, else 0
3     with torch.no_grad():
4         self.spike_sum += y.sum()       # count of fired elements
5         self.elem_count += y.numel()    # total elements seen
6     return y
7
8 def rate(self):
9     # fraction of elements that fired
10    return (self.spike_sum / self.elem_count).item()
```

`mean_firing_rate()` averages `rate()` across all SpikeAct instances in the network. After 300 epochs on the A100:

$$r_{\text{MN10}} = 0.2627, \quad r_{\text{MN40}} = 0.2445.$$

Step 2 — Energy model

On neuromorphic hardware, SNN energy is proportional to the number of *spike events* (synaptic operations, SOPs). Each fired spike triggers an *accumulate* (AC); neurons that do not fire

consume no dynamic energy:

$$E_{\text{SNN}} = r \cdot N_{\text{syn}} \cdot E_{\text{AC}}.$$

A comparable ANN performs a *multiply-accumulate* (MAC) for every activation, every layer, every inference:

$$E_{\text{ANN}} = N_{\text{syn}} \cdot E_{\text{MAC}}.$$

Step 3 — Conservative estimate (firing rate only)

The $3.8\times$ figure in the abstract uses the simplest bound: assume $E_{\text{AC}} \approx E_{\text{MAC}}$ and count only the sparsity gain from the firing rate:

$$\frac{E_{\text{SNN}}}{E_{\text{ANN}}} = \frac{r \cdot E_{\text{AC}}}{E_{\text{MAC}}} \approx r = 0.263 \implies \text{savings} = \frac{1}{r} = \frac{1}{0.263} \approx \mathbf{3.8\times}.$$

This is intentionally *conservative*. It ignores the second source of savings below.

Step 4 — Full estimate (sparsity $\times$ operation type)

On 45 nm CMOS, an AC costs ~ 0.9 pJ while a MAC costs ~ 4.6 pJ [7] — a factor of $\approx 5\times$. The full energy ratio is therefore:

$$\frac{E_{\text{SNN}}}{E_{\text{ANN}}} = r \cdot \frac{E_{\text{AC}}}{E_{\text{MAC}}} = 0.263 \times \frac{0.9}{4.6} \approx 0.051 \implies \text{full savings} \approx \mathbf{19\times}.$$

Table 6: Energy efficiency breakdown (MN10, $r = 0.2627$, 45 nm CMOS).

Source of savings	Factor	Cumulative
Spike sparsity ($1/r$)	$3.80\times$	$3.80\times$
AC vs. MAC ($E_{\text{MAC}}/E_{\text{AC}}$)	$5.1\times$	$\sim 19\times$
ASP early exit (4/3.89 chunks skipped)	$1.03\times$	$\sim 20\times$
Reported (conservative)	$3.8\times$ (sparsity only)	
Full theoretical	$\sim 20\times$ (on neuromorphic HW)	

The $3.8\times$ figure is reported throughout the paper because it is hardware-agnostic and verifiable directly from the training logs. The $\sim 20\times$ figure applies specifically to neuromorphic hardware (Loihi 2, SpiNNaker) where AC and MAC are physically distinct operations; on a GPU both are implemented as FP32/BF16 tensor operations at the same cost.

8. Comparison with Literature

Table 7: Comparison against published methods on ModelNet40. Our method is the only SNN achieving $> 89\%$.

Method	Type	MN10 Acc	MN40 Acc
PointNet [1]	ANN	—	89.2%
PointNet++ [2]	ANN	—	91.9%
PointTransformer [3]	ANN	—	93.7%
DGCNN [4]	ANN	—	92.9%
Spiking PointNet [5]	SNN	—	84.3%
SPM + ASP (ours)	SNN	93.28%	89.10%

* Literature numbers from published papers. Ours from A100 training run (300 epochs, MiKTeX compiled). SNN results are limited in literature; SPM+ASP closes $> 5\text{pp}$ gap vs prior SNN SOTA on MN40.

9. Discussion

9.1 Why ASP Boosts Accuracy on MN10

The slice policy acts as an implicit *spatial ensemble*: averaging chunk-level predictions reduces variance, and skipping uninformative chunks reduces noise. On the simpler 10-class task, the benefit outweighs the information loss from skipping.

9.2 Why ASP Slightly Underperforms on MN40

With 40 fine-grained classes, subtle inter-class differences (desk vs. table, sofa vs. chair) may be encoded in the chunks skipped by ASP. The marginal compute saving (-0.40pp , -0.16 chunks) is still a favourable trade-off for edge deployment.

9.3 Limitations

- **torch.compile incompatibility:** FPS has a Python for-loop of 128 iterations, causing 128 graph breaks and hanging Triton compilation. Disabled; native eager mode is used instead.
- **Theoretical vs. actual energy:** measurements are on GPU; actual SNN energy savings are only realised on neuromorphic hardware (Loihi 2, SpiNNaker).
- **Data loading:** ModelNet ‘.off’ mesh parsing via trimesh adds ~ 10 minutes on first epoch but negligible thereafter.

10. Conclusion

We demonstrated that a spiking neural network equipped with selective state-space mixing (Mamba-lite), adaptive spatial slicing (ASP), and knowledge distillation from an ANN teacher can match or exceed ANN accuracy on 3D point cloud classification while operating at substantially lower spike firing rates.

Summary of contributions:

- SPM: first Mamba-SSM-based SNN for 3D point clouds.
- ASP: learnable spatial chunk selection with early exit, reducing compute by up to 15%.
- KD: SNN student surpasses its ANN teacher on ModelNet10 (93.28% vs. 91.19%).
- Energy: $3.8\times$ theoretical savings from 26.3% firing rate.

Future work includes deployment on Loihi 2 neuromorphic hardware for real energy measurement, extending ASP to 6-DoF object detection (ScanNet), and replacing FPS with a compile-friendly CUDA kernel.

References

- [1] Qi et al., “PointNet: Deep Learning on Point Sets,” CVPR 2017.
- [2] Qi et al., “PointNet++: Deep Hierarchical Feature Learning,” NeurIPS 2017.

- [3] Zhao et al., “Point Transformer,” ICCV 2021.
- [4] Wang et al., “Dynamic Graph CNN for Learning on Point Clouds,” TOG 2019.
- [5] Ren et al., “Spiking PointNet: Spiking Neural Networks for Point Clouds,” NeurIPS 2023.
- [6] Hinton et al., “Distilling the Knowledge in a Neural Network,” NeurIPS 2015 Deep Learning Workshop.
- [7] Horowitz, “1.1 Computing’s Energy Problem (and what we can do about it),” ISSCC 2014. (45nm CMOS: MAC $\approx$ 4.6 pJ, AC $\approx$ 0.9 pJ.)